Xplore - Technical Overview

Un'app Android per il tracciamento e la documentazione dei viaggi

Sviluppatore: Raffaele Melluso
Corso: Programmazione Mobile

Architettura e Funzionalità Locale

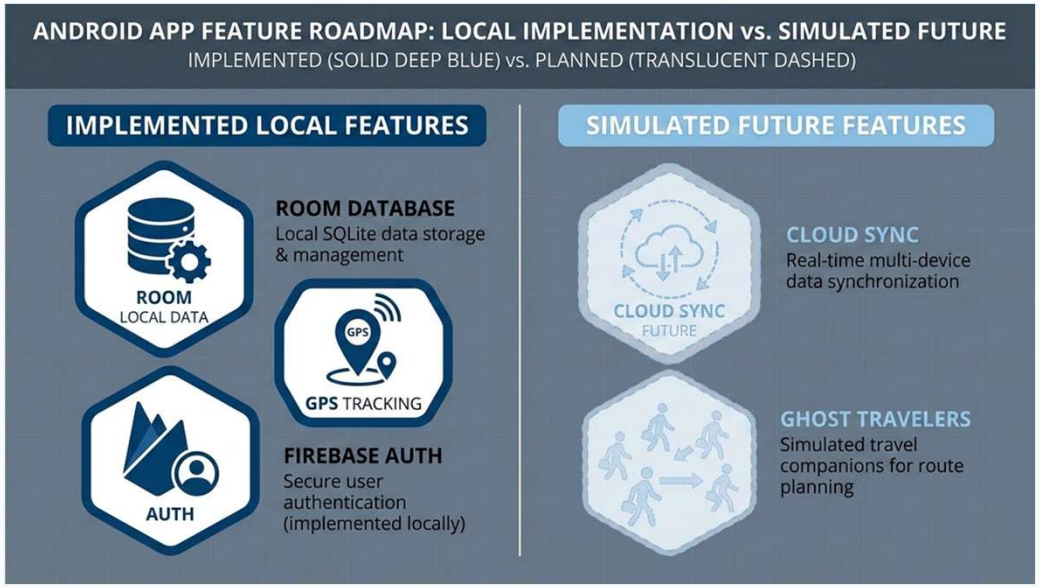
Xplore

Xplore documenta e traccia i viaggi combinando localizzazione, contapassi e mappe.

La versione finale è prevalentemente locale: i dati dei viaggi, dei punti di interesse (POI) e delle statistiche sono interamente gestiti tramite **Room**.

Firebase Authentication gestisce l'identità dell'utente in sicurezza.

Per dimostrare l'interazione social senza un backend remoto, l'app utilizza dati dimostrativi locali simulando altri viaggiatori, denominati **ghost**, con percorsi e visite generate internamente.

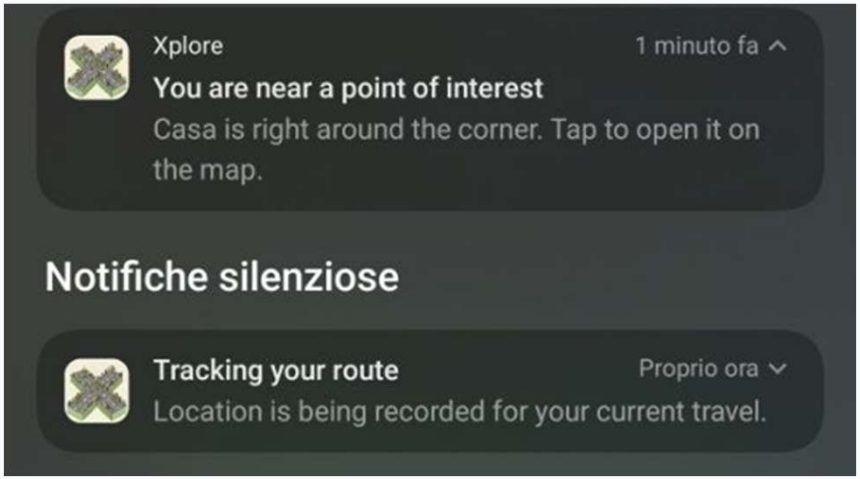


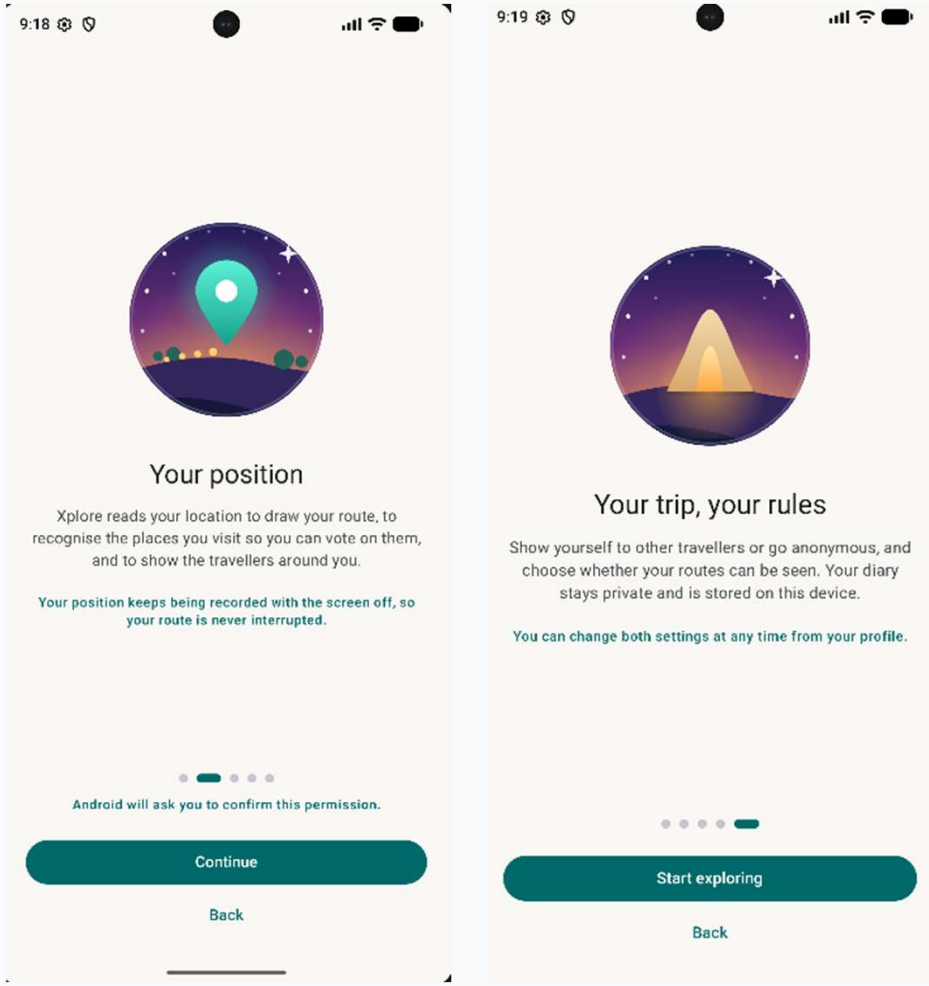
Aree Funzionali dell'Applicazione

Xplore

Registrazione Percorsi in Background

Tracciamento continuo dei tragitti mediante GPS e sensore contapassi gestiti in background. Notifica persistente con possibilità di fermare il servizio quando si vuole



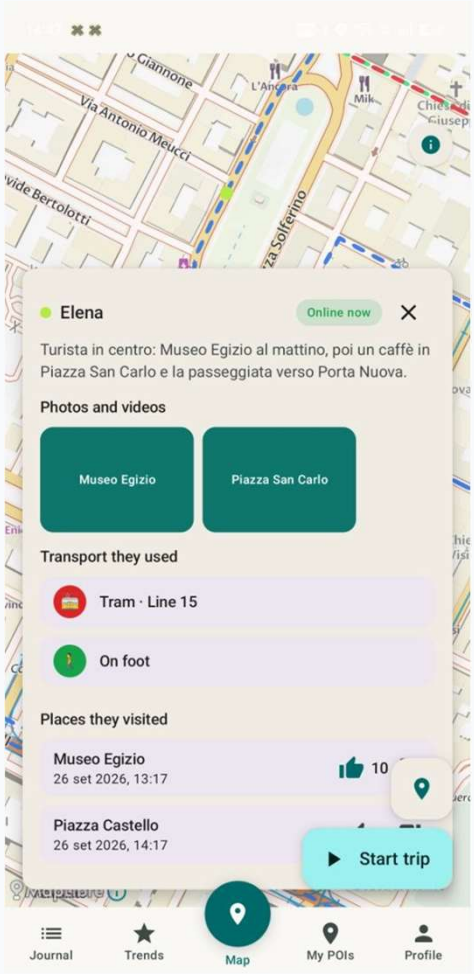


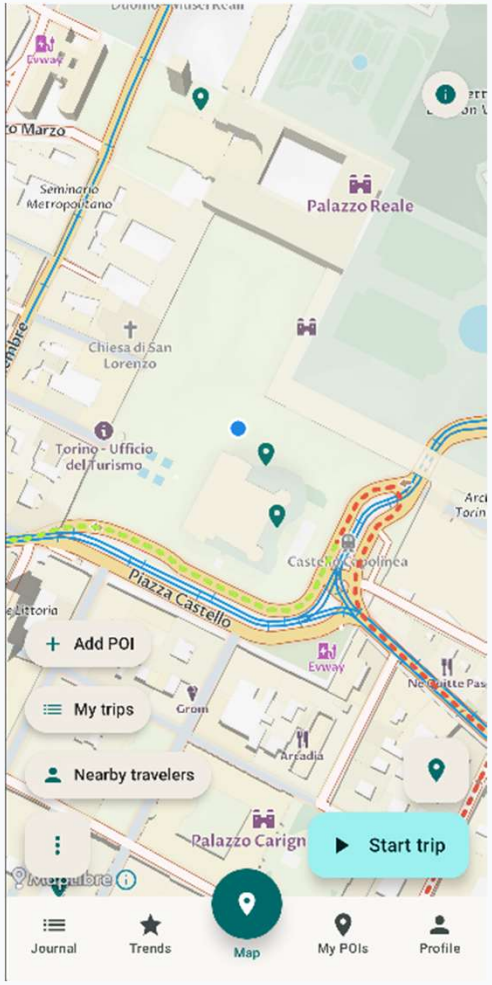
Aree Funzionali dell'Applicazione

Xplore

Mappa e Ghost Simulati

Visualizzazione interattiva della propria posizione, del percorso fatto, dei punti di interesse (POI) e dei ghost simulati. I ghosts hanno percorsi, punti di interesse, voti e mezzi presi.





Mappa: MapLibre Native e OpenFreeMap

Rendering Mappe in Compose

- **Zero API key, zero costi:** tile vettoriali OpenFreeMap senza token da proteggere né quote da monitorare.
- **Compose con View:** la MapView è ospitata in un AndroidView, quindi Compose non ricompone mai la mappa.
- **Rendering:** percorso, POI e ghost sono sorgenti GeoJSON aggiornate solo quando lo stato cambia.
- **Ciclo di vita gestito:** la mappa viene distrutta al rilascio per liberare le risorse ed evitare leak.

```
const val STYLE_URL =
  "https://styles.maptoolkit.org/street.json"

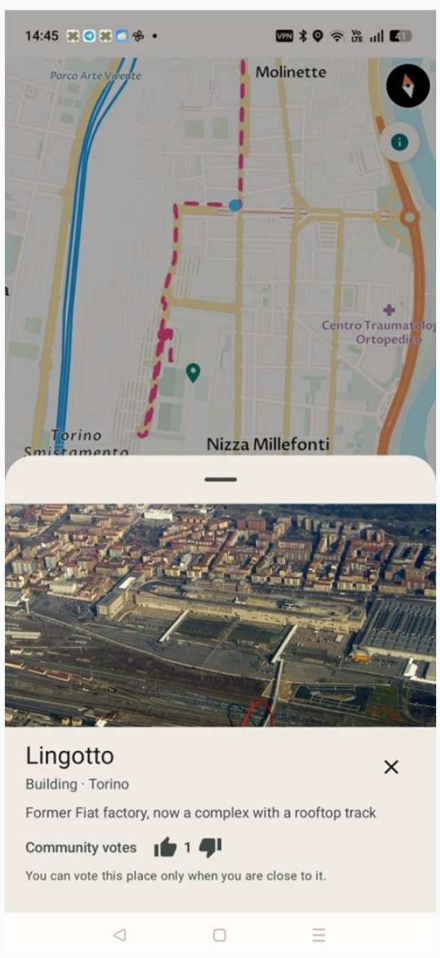
AndroidView(
  factory = { ctx ->
    MapLibre.getInstance(ctx)
    MapView(ctx).apply {
      getMapAsync { map ->
        map.setStyle(
          Style.Builder().fromUri(STYLE_URL)
        ) { style ->
          overlays = MapOverlays(map, style)
        }
      }
    }
  },
  update = { mapView ->
    overlaysRef.value?.update(uiState)
  },
  onRelease = { mapView ->
    mapView.onDestroy()}
)
```

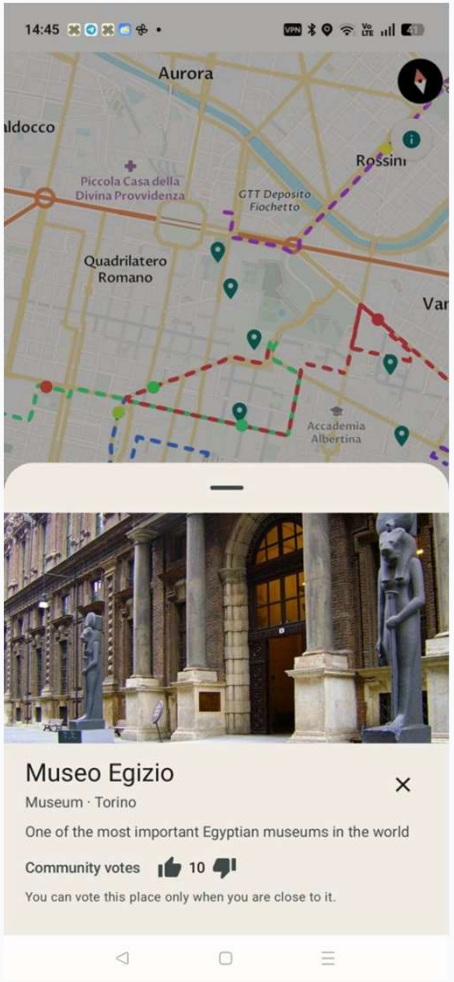
Aree Funzionali dell'Applicazione

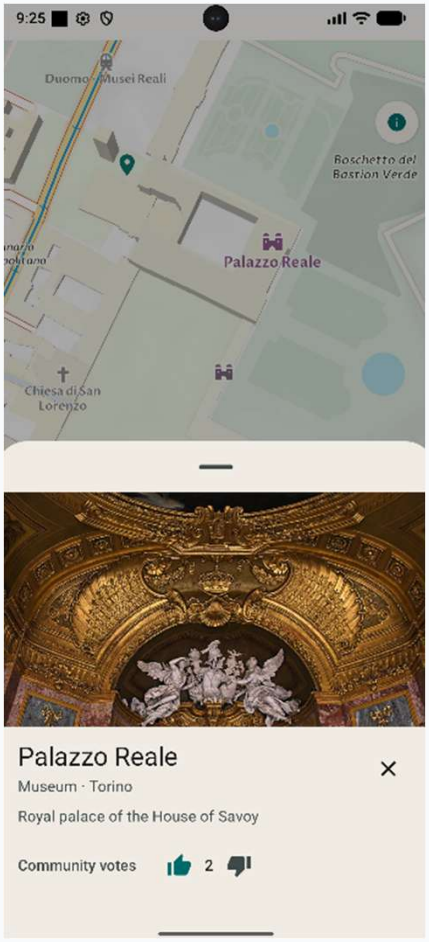
Xplore

Sistema di Prossimità

Modulo di notifica automatica in tempo reale per segnalare i POI nelle vicinanze dell'utente.





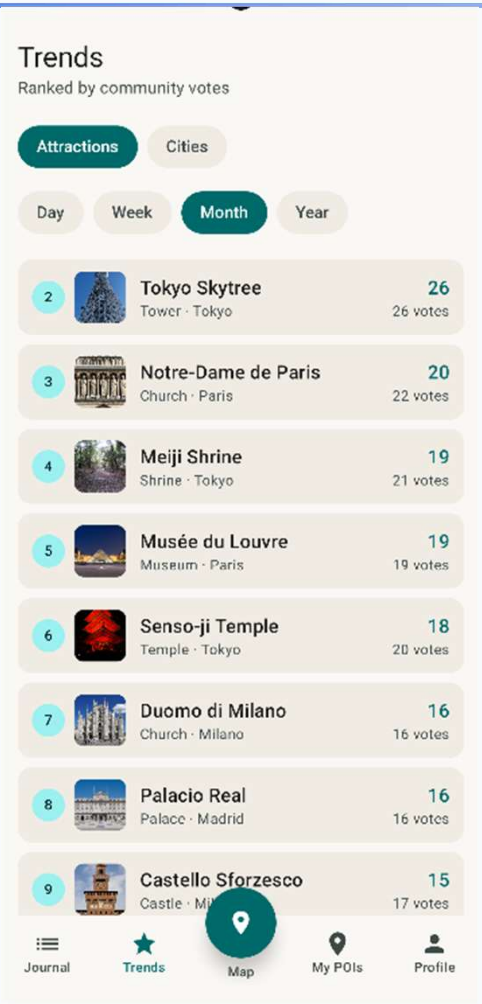


Aree Funzionali dell'Applicazione

Xplore

Statistiche e Trend Temporali

Calcolo di metriche aggregate e analisi dei trend (giorno, settimana, mese, anno) basati sui voti espressi per i POI. Si possono filtrare per attrazioni o per città da visitare





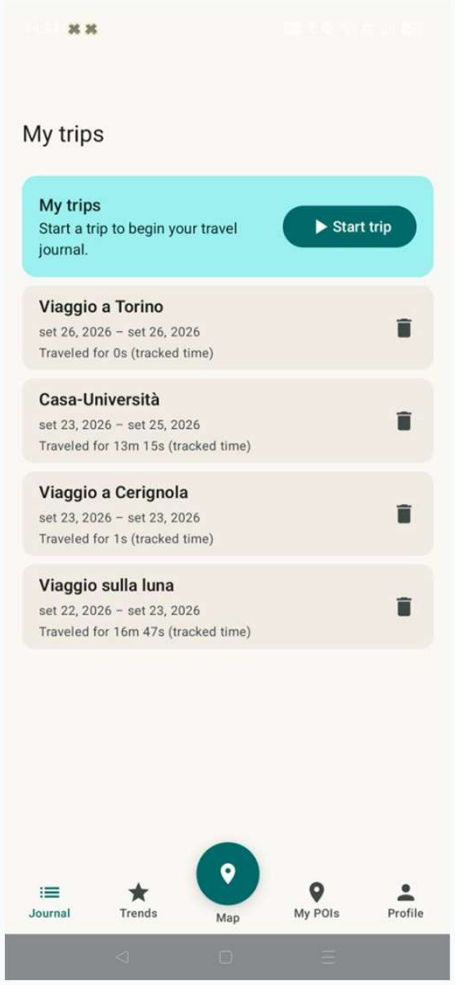


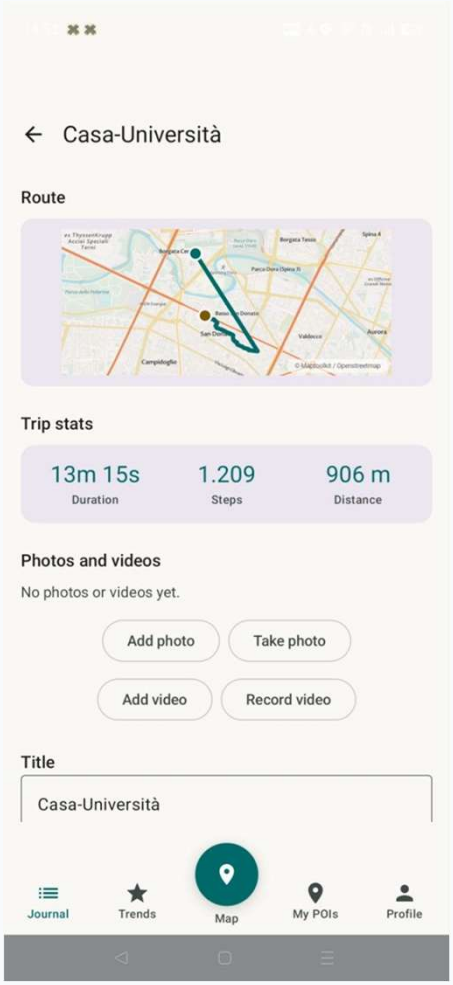
Aree Funzionali dell'Applicazione

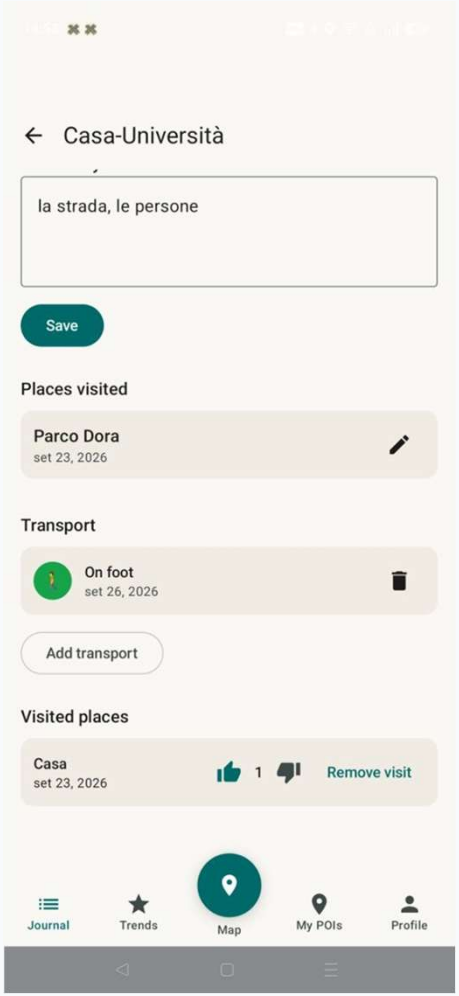
Xplore

Libretto Viaggi

Sezione per conservare i ricordi di viaggio, con supporto al salvataggio di foto, dettagli sui trasporti, percorsi, statistiche di viaggio e punti di interesse visitati







Architettura: MVVM & Repository Pattern

Xplore

UI Layer

Jetpack Compose

Inoltra gli eventi dell'utente ai ViewModel.

Osserva lo stato reattivo.

State & Logic

ViewModel

Gestisce la logica di presentazione e mantiene lo stato della UI.

Comunica con i Repository.

Abstraction

Repository

Astrae e coordina le sorgenti dati sottostanti.

Fornisce un'interfaccia pulita verso l'alto.

Data Sources

Room & DataStore

Persistenza Locale

Firebase

Autenticazione Cloud

Sensori

Foreground Service

Isolamento e Disaccoppiamento

La **UI non accede mai direttamente** al database o ai sensori. La logica applicativa rimane perfettamente isolata, garantendo testabilità, modularità e manutenibilità del codice.

Iniezione delle Dipendenze: AppContainer

Architettura: Composition Root

- **Costruzione centralizzata:** un singleton collega database, repository e autenticazione, la UI non tocca mai DAO o sorgenti dati.
- **Singleton thread-safe:** il database nasce una sola volta, con double-checked locking su un campo volatile.
- **Isolamento per-utente:** ogni repository riceve un userIdProvider, così i dati sono separati per account Firebase.

```
object AppContainer {
  @Volatile private var db: AppDatabase? = null

  fun database(context: Context): AppDatabase =
    db ?: synchronized(this) {
      db ?: Room.databaseBuilder(
        context.applicationContext,
        AppDatabase::class.java, "xplore.db"
      ).addMigrations(
        MIGRATION_5_6, // v5-v6 ... v11-v12
        MIGRATION_11_12
      ).build().also { db = it }
    }

  fun mapRepository(context: Context) =
    MapRepositoryImpl(
      locationFlow = LocationUpdates.samples,
      poiDao = database(context).poiDao(),
      ghostSimulator = GhostFactory(),
      userIdProvider = { currentUserId() }
    )
}
```

Architettura ViewModel & Stato UI

Xplore

Architettura dei Componenti

ViewModel Dedicati

Ogni macrofunzionalità è isolata in un proprio ViewModel responsabile della logica di business:

MapViewModel • TravelJournalViewModel • TrendViewModel • ProfileViewModel • LoginViewModel

Flusso Dati Reattivo

StateFlow & Coroutine

I ViewModel espongono stati immutabili tramite **StateFlow** per garantire la consistenza dei dati.

Tutte le operazioni asincrone e in background sono eseguite in sicurezza utilizzando **viewModelScope**.

Rendering Dichiarativo

UI in Jetpack Compose

La UI raccoglie lo stato esposto dal ViewModel e si ricompone reattivamente ad ogni modifica.

L'interfaccia reagisce automaticamente ai cambiamenti di stato senza aggiornamenti manuali.

Unica Base di Verità (Source of Truth)

Stato Coerente

Garantisce che la schermata non debba coordinare sorgenti indipendenti, ma si limiti a osservare un unico stato coerente.

Elimina gli disallineamenti di stato e semplifica il testing.

Persistenza Reattiva e State Flow

Xplore

MapViewModel

Un esempio chiave dell'architettura reattiva è il **MapViewModel** , che orchestra molteplici sorgenti di dati in tempo reale.

Utilizza l'operatore **combine** per unire reattivamente:

- Posizione attuale
- Percorsi registrati
- Punti di interesse (POI)
- Ghost ed elementi tracciati
- Conteggio passi

Tutti i flussi confluiscono in un unico stato aggiornato istantaneamente nella schermata Compose.

Flusso di Tracciamento in Background

Xplore

1. Avvio Foreground Service

Inizializzazione di TrackingService con notifica persistente per garantire l'esecuzione continua anche con app in background o schermo spento.

2. Lettura Sensori (GPS e Contapassi)

Ricezione continua della posizione geografica dal provider Android e contestuale lettura dei dati dal sensore contapassi dispositivo.

3. Filtraggio delle Coordinate

Validazione e pulizia delle coordinate ricevute secondo criteri definiti di distanza minima, intervallo temporale e livello di accuratezza.

4. Salvataggio Coroutine in Room

Esecuzione delle operazioni di persistenza su database Room tramite Coroutine su un gestore di scope dedicato del servizio, slegato dal ciclo di vita UI.

Flusso di Tracciamento in Background

Xplore

Gestione Lifecycle Service

onStartCommand

All'avvio verifica richieste di interruzione; altrimenti si promuove a **foreground service**, avvia il riconoscimento attività e apre la sessione. **START_STICKY ricrea il servizio se terminato.**

```
override fun onStartCommand(
    intent: Intent?,
    flags: Int,
    startId: Int
): Int {
    if (
        intent?.action == ACTION_STOP ||
        (intent == null && wasStopRequested())
    ) {
        requestStop()
        return START_NOT_STICKY
    }

    clearStopRequest()
    startAsForeground()

    activityRecognitionMonitor.start()
    observeTrackingState()

    serviceScope.launch {
        travelRepository.startSession()
    }

    requestCurrentLocation()

    return START_STICKY
}
```

Elaborazione Sensori & Tempo

onSensorChanged

Il sensore fornisce un contatore cumulativo di passi. Il timestamp viene convertito in **tempo UTC** per una registrazione accurata.

- **Garantisce la corretta sequenza temporale**
- **Gestisce elaborazioni in ritardo nel callback**

```
override fun onSensorChanged(event: SensorEvent) {
    val value = event.values.firstOrNull()?.toLong()
        ?: return

    val readingUtcMillis =
        SensorTime.wallClockUtcMillis(
            sensorNanos = event.timestamp,
            wallClockNowMillis =
                System.currentTimeMillis(),
            elapsedRealtimeMillis =
                SystemClock.elapsedRealtime()
        )

    serviceScope.launch {
        stepRepository.onStepCount(
            value,
            readingUtcMillis
        )
    }
}
```

Ottimizzazione Batteria: TrackingMode

Xplore

Gestione Energetica del Service

- **Tre profili di campionamento:** la precisione richiesta al provider segue lo stato reale dell'app, non un valore fisso.
- **ACTIVE:** viaggio in registrazione o app a schermo: GPS ad alta precisione per un percorso fedele.
- **IDLE:** nessun viaggio e app in background: provider economico con batching delle posizioni.
- **PAUSED:** utente fermo secondo Activity Recognition: nessuna posizione richiesta; le transizioni di contesto risvegliano il tracciamento.
- **Funzione pura, testata:** la decisione è unit-testabile senza dispositivo; lo stato UNKNOWN degrada in sicurezza a IDLE.

```
enum class TrackingMode {
  ACTIVE, IDLE, PAUSED;

  companion object {
    // Funzione pura dei 3 segnali:
    // unit-testabile senza device
    fun of(
      tripActive: Boolean,
      appInForeground: Boolean,
      motion: MotionState = UNKNOWN
    ): TrackingMode = when {
      tripActive || appInForeground ->
        ACTIVE
      motion == MotionState.STILL ->
        PAUSED
      else -> IDLE // UNKNOWN -> sicuro
    }
  }
}
```

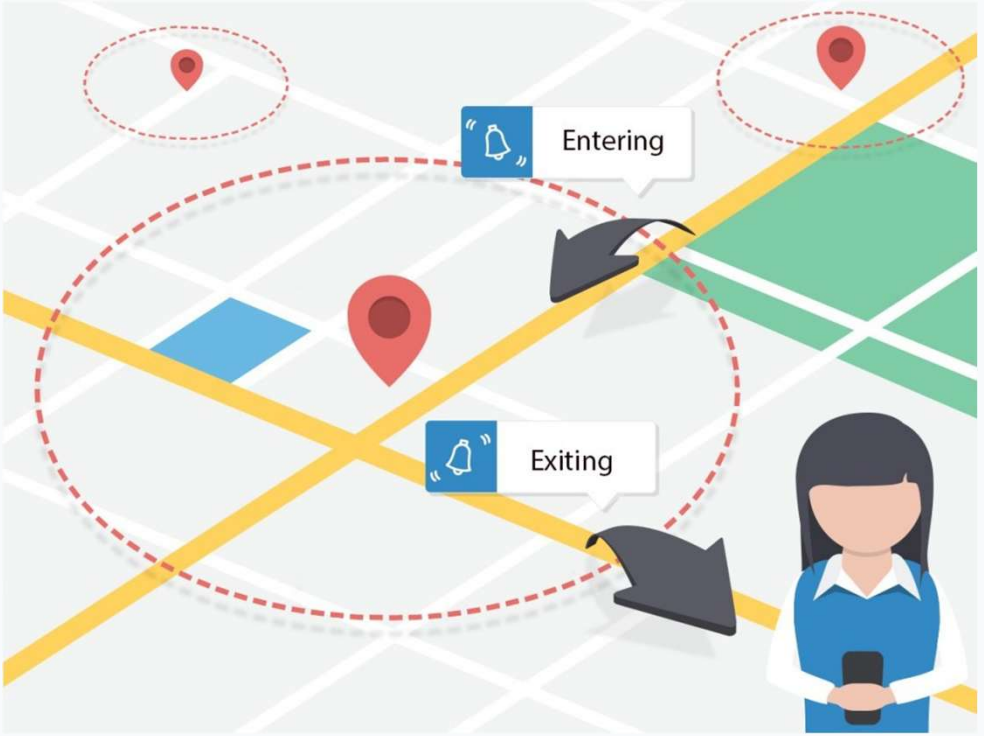
Rilevamento Punti di Interesse (POI)

Xplore

Implementazione Custom

Il rilevamento dei punti di interesse vicini **non utilizza l'API Android Geofencing**.

- **Calcolo Autonomo:** L'applicazione calcola autonomamente la distanza tra le coordinate dell'utente e i POI salvati in Room.
- **Condizioni di Attivazione:** Il POI è considerato vicino se la distanza è inferiore al raggio configurato e l'accuratezza del GPS è sufficiente.
- **Anti-Duplicazione:** Per evitare notifiche duplicate, viene memorizzato l'identificativo dell'ultimo POI notificato, riattivando l'avviso solo uscendo e rientrando nell'area.



Rilevamento Punti di Interesse (POI)

Xplore

Viewmodel e Haversine

Il ViewModel combina la posizione corrente con l'elenco dei POI. La funzione PoiProximity.near filtra i punti che si trovano all'interno del raggio configurato. Tra quelli trovati viene scelto il più vicino usando la distanza calcolata con la formula di Haversine.

```
private val nearbyPoiFlow = combine(
    mapRepository.observeCurrentLocation(),
    mapRepository.observePois()
) { location, pois ->
    if (location == null || pois.isEmpty()) {
        null
    } else {
        PoiProximity.near(
            location.latitude,
            location.longitude,
            pois,
            config.poiVisitRadiusMeters
        ).minByOrNull { poi ->
            GeoUtils.haversineMeters(
                location.latitude,
                location.longitude,
                poi.lat,
                poi.lon
```

Timerizzazione POI

La scheda non viene aperta immediatamente: viene applicato un ritardo di cinque secondi. Inoltre, l'insieme shownGeofencePoilds impedisce di mostrare più volte lo stesso POI durante il viaggio corrente.

```
val inFence =
    mapReady &&
    poi == null &&
    selectedGhost == null &&
    nearby != null

LaunchedEffect(nearby?.id, mapReady) {
    if (!inFence ||
        nearby == null ||
        nearby.id in uiState.shownGeofencePoiIds
    ) {
        autoSheetVisible = false
    } else {
        autoSheetVisible = false
        delay(5_000)

        if (nearby.id !in uiState.shownGeofencePoiIds) {
            viewModel.markGeofenceShown(nearby.id)
            autoSheetVisible = true
        }
    }
```

PoiDetailSheet

La stessa PoiDetailSheet viene utilizzata sia per il tap manuale sul marker sia per l'apertura automatica causata dal geofencing.

```
val shownPoi = poi ?: autoPoi

if (shownPoi != null) {
    PoiDetailSheet(
        poi = shownPoi,
        myVote = uiState.poiVotes[shownPoi.id] ?: 0,
        canVote = shownPoi.id == uiState.nearbyPoi?.id,
        onVotePoi = { id, voteValue ->
            viewModel.votePoi(id, voteValue)
        },
        onDismiss = {
            viewModel.markGeofenceShown(shownPoi.id)
        }
    )
}
```

Persistenza Locale: Room e DataStore

Xplore

Room

Database Relazionale Completo

Gestisce la persistenza strutturata per entità complesse e dati ad alta densità relazionale:

- **Viaggi:** itinerari e informazioni generali
- **Percorsi GPS:** tracciamento e coordinate
- **Passi giornalieri:** metriche e attivita fisiche
- **Città & POI:** punti di interesse e località
- **Voti:** valutazioni e recensioni utente

Jetpack DataStore

Chiave-Valore Leggero

Impiegato per la memorizzazione di preferenze utente semplici e persistenti:

- **Tema dell'app:** preferenze d'interfaccia
- **Modalità anonima:** impostazioni di privacy
- **Visibilità percorsi:** stato di condivisione
- **Stato onboarding:** completamento guida iniziale

Integrazione Firebase:
Le preferenze sono associate all'identificativo Firebase per supportare il cambio account in modo trasparente.

Persistenza Locale: Room e DataStore

Xplore

Room

La classe rappresenta una tabella Room contenente i voti sui POI. La foreign key collega ogni voto al relativo POI e onDelete = CASCADE elimina automaticamente i voti quando il POI viene cancellato. L'indice univoco su poiId e voterUserId garantisce che ogni utente abbia un solo voto per ciascun POI.

```
@Entity(
    tableName = "poi_votes",
    foreignKeys = [
        ForeignKey(
            entity = PoiEntity::class,
            parentColumns = ["id"],
            childColumns = ["poiId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [
        Index(
            value = ["poiId", "voterUserId"],
            unique = true
        )
    ]
)
data class PoiVoteEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0L,
    val poiId: Long,
    val voterUserId: String,
    val voteValue: Int,
    val votedAtUtcMillis: Long
)
```

Datastore

Il DataStore viene utilizzato per salvare preferenze semplici, come modalità anonima, visibilità dei percorsi, avatar e tema. I dati sono esposti come Flow, quindi la UI riceve automaticamente gli aggiornamenti.

```
val preferences: Flow<UserPreferences> =
    store.data
        .map { prefs ->
            UserPreferences(
                anonymous = prefs[anonymousKey()] ?: false,
                showTravelPaths =
                    prefs[showTravelPathsKey()] ?: true,
                avatarUri = prefs[avatarKey()],
                themeMode =
                    prefs[themeModeKey()]?.let { value ->
                        ThemeMode.entries.firstOrNull {
                            it.name == value
                        }
                    } ?: ThemeMode.SYSTEM
            )
        }
```

FirebaseAuthRepository

Xplore

FirebaseAuthRepository gestisce l'identità dell'utente nascondendo i dettagli dell'SDK Firebase alla UI.

- **Autenticazione Flessibile:** Supporta la registrazione e il login tramite e-mail/password e Google Sign-In.
- **Integrazione Coroutine:** Converte le operazioni asincrone di Firebase in funzioni suspend per le coroutine.
- **Gestione Errori:** Traduce gli errori del provider in errori del dominio applicativo.

Gestione dello Stato Reattivo

"Lo stato della sessione viene esposto reattivamente tramite callbackFlow, permettendo alla MainActivity di modificare automaticamente la navigazione."

GhostFactory & Simulazione Social

Xplore

Simulazione Locale dei Dati

La versione finale non possiede un backend remoto per la sincronizzazione dei viaggi tra utenti reali. La componente social è simulata localmente tramite una **GhostFactory**.

Proprietà dei Ghost

I ghost possiedono identificativi, percorsi, visite ai POI e voti generati internamente.

Vantaggi dell'Architettura

Questa scelta architetturale permette di dimostrare le funzionalità di interazione, filtraggio e visualizzazione sulla mappa, riducendo la complessità e mantenendo la logica pronta per una futura integrazione server.



La factory è statica, con percorsi effettuati generati da intelligenza artificiale

MapViewModel & Gestione dello Stato

Xplore

Il codice più significativo risiede nel **MapViewModel**, dove l'operatore **combine** di Kotlin unisce molteplici Flow.

ARCHITETTURA REATTIVA • PATTERN COMBINE

Il ViewModel sincronizza la posizione corrente, i percorsi tracciati, i punti di interesse dal database, i ghost simulati e i passi del sensore.



Questa combinazione genera un singolo **MapUiState non mutable**. La schermata osserva esclusivamente questo stato, garantendo un **aggiornamento fluido e reattivo** della mappa a ogni variazione dei dati sottostanti.

MapViewModel & Gestione dello Stato

Xplore

- Il **MapViewModel** raccoglie dati provenienti da diversi repository: posizione dell'utente, percorso, POI, ghost e viaggi.
- La funzione **combine** aspetta i valori più recenti di tutti i flussi e costruisce un unico oggetto MapUiState.
- La UI osserva soltanto questo stato e si aggiorna automaticamente quando cambia uno dei dati. In questo modo la logica applicativa rimane nel ViewModel e non direttamente nella schermata.

```
val uiState: StateFlow<MapUiState> =
    combine(
        mapRepository.observeCurrentLocation(),
        pathFlow,
        mapRepository.observePois(),
        mapRepository.observeGhosts(),
        travelsAndCitiesFlow
    ) { location, path, pois, ghosts, travelsAndCities ->

        val (travels, cities) = travelsAndCities

        MapUiState(
            userLocation = location,
            pathPoints = path,
            pois = pois,
            ghosts = ghosts,
            travels = travels,
            travelCities = cities,
            displayedTravelId = selectedTravelId.value,
            isLoading = false
        )
    }.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = MapUiState(isLoading = true)
    )
```

Strategia di Validazione e Testing

Xplore

LOGICA DI DOMINIO

JUnit & Mockito

Isolamento completo della business logic mediante test unitari dedicati:

- **Calcolo distanza:** algoritmo verificato per evitare il doppio conteggio tra GPS e pedometro.
- **Segmentazione:** corretta gestione del percorso in caso di interruzione del segnale.

FLUSSI ASINCRONI

Coroutine Testing

Strumenti dedicati per la gestione e la sincronizzazione del codice asincrono:

- **Esecuzione controllata:** test deterministici sulle coroutine dello stato di tracciamento.
- **Gestione thread:** isolamento delle operazioni I/O dai Dispatcher di background.

PERSISTENZA LOCAL

In-Memory Room DB

Test strumentali eseguiti in memoria temporanea RAM:

- **Validazione DAO:** verifica diretta di query, inserimenti e transazioni del database.
- **Sicurezza dati:** esecuzione rapida senza alterare mai i dati persistenti reali.

Persistenza Locale: Room e DataStore

Xplore

Room

Il test verifica che il metodo startTravelFromMap del ViewModel inoltri correttamente l'azione al TravelRepository.

- mock crea un repository simulato;
- runTest permette di testare codice basato su coroutine;
- advanceUntilIdle attende il completamento delle coroutine;
- verify controlla che il metodo sia stato chiamato.

```
@Test
fun startTravelFromMap_startsTravel() =
    runTest(dispatcher) {
        val travelRepo = mock<TravelRepository>()

        val viewModel = MapViewModel(
            repository,
            mock<PoiRepository>(),
            voteRepository,
            stepRepository(),
            travelRepo
        )

        backgroundScope.launch {
            viewModel.uiState.collect { }
        }

        advanceUntilIdle()

        viewModel.startTravelFromMap()
        advanceUntilIdle()

        verify(travelRepo).startTravel()
    }
```

Datastore

Il database Room viene creato in memoria e chiuso al termine del test. In questo modo è possibile testare DAO, query, vincoli e relazioni senza modificare il database reale dell'app.

```
@Test
fun visitUniqueConstraint_ignoresDuplicateVisitForSameTravelAndPoi() =
    runBlocking {
        val travelId =
            database.travelDao().insert(travel())

        val poiId =
            database.poiDao().insert(poi())

        val firstId =
            database.travelPoiVisitDao().insert(
                visit(
                    travelId,
                    poiId,
                    100L
                )
            )

        val duplicateId =
            database.travelPoiVisitDao().insert(
                visit(
                    travelId,
                    poiId,
                    200L
                )
            )

        val visits =
            database.travelPoiVisitDao()
                .visitsForTravel(travelId)
                .first()

        assertNotEquals(0L, firstId)
        assertEquals(-1L, duplicateId)
        assertEquals(1, visits.size)
        assertEquals(
            100L,
            visits.single().visitedAtUtcMillis
        )
    }
```

Limiti Attuali e Sviluppi Futuri

Xplore

ARCHITETTURA LOCALE

Limiti Attuali

L'applicazione presenta limiti deliberati coerenti con gli obiettivi del corso.

Assenza di Sincronizzazione Remota

Manca una sincronizzazione remota dei dati applicativi.

Assenza di Moderazione POI

Manca una moderazione server-side dei POI.

Dati Community Locali

I dati della community sono completamente predisposti in locale.

BACKEND CLOUD & REAL-TIME

Sviluppi Futuri

Evoluzioni architetturali e funzionali previste per le prossime release.

Backend Remoto

Integrazione di un backend remoto per sostituire i ghost con utenti reali.

Chat Privata e di Gruppo

Aggiunta di un sistema di messaggistica privata o di gruppo.

Live Sharing Percorsi

Condivisione in tempo reale dei percorsi tra utenti.

Architettura e Stack Tecnologico

Xplore

Architettura MVVM e Reattività

Xplore dimostra una padronanza completa dello stack moderno Android. L'app separa rigorosamente l'interfaccia dalla logica di business applicando l'architettura MVVM.

L'integrazione di Jetpack Compose, Room e Kotlin Flow crea un ecosistema totalmente reattivo, garantendo un'interfaccia utente sempre aggiornata e una gestione fluida dello stato.

TRACCIAMENTO ASINCRONO

Garantito da Foreground Service e Coroutine, ottimizzando l'uso dei sensori hardware.

RISULTATO

Un'applicazione robusta, scalabile e pronta per future evoluzioni.

Grazie per l'Attenzione!

Sono a disposizione per domande e approfondimenti sul progetto Xplore.